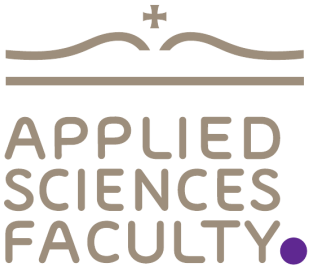
C++ — лекція 5. Динамічна пам'ять – 1





©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)

Керування динамічною памяттю

- В C++ виділення пам'яті здійснюється за допомогою оператора **new**[1]:

  double *ddp = **new double**;

  — виділення пам'яті під одну змінну типу double.
- Для виділення масивів використовується спеціальна версія new:

  double *darr = **new double** [15];

  — виділення пам'яті під 15 змінних типу double.
- Виділену пам'ять слід завжди звільняти, коли вона більше не потрібна! Для звільнення служать оператори **delete** і **delete[]**.
- Пам'ять, виділену new, слід **завжди** звільняти виключно delete, а виділену new [] — виключно delete[]:

  delete ddp;
  delete [] darr;

- Звертання до звільненої пам'яті є дуже грубою помилкою!
- Зате видалення нульового вказівника просто не робить нічого.

[1] Тут є деякі нюанси. Зокрема, різниця між оператором new і виразом new. Див, наприклад, http://habrahabr.ru/post/185662/

Вираз new та оператор new

- Існує тонкий нюанс роботи new (та delete).
- Вираз new:

  double *dwa = **new double** [Arr_size];

  виділяє пам'ять, за допомогою **оператора new**, після чого, за потреби, викликає необхідні конструктори.
- Аналогічно поводиться і вираз delete — він викликає, за потреби, деструктори, а потім, за допомогою **оператора** delete, звільняє пам'ять.

placement new

Існує варіант виразу new, який використовує вже виділену пам'ять[2], просто конструює у ній об'єкт та повертає переданий йому вказівник.

```
class MyClass {
public:
    MyClass() {
        std::cout << "constructed at address: " << this << endl;
    }
    ~MyClass() {
        std::cout << "Destroyed: " << this << endl;
    }
};

    std::cout << "Ordinary new: ";
    MyClass * p1 = new MyClass;

    std::cout << "Placement new: ";
    new (p1) MyClass;
```

Версія new[] працює аналогічно.

[2] Тобто, не викликає оператор new!

Обробка помилок виділення пам'яті

Трапляються ситуації, коли виділити пам'ять неможливо. Наприклад, вільна пам'ять закінчилася.

- Оператор new має два способи повідомити про це.
- Перший — згенерувати виключення `bad_alloc`.
- Другий — повернути нульовий вказівник.
- Згідно стандарту, new генерує виключення, new(nothrow) повертає нуль, у випадку, коли не може виділити пам'ять. На практиці конкретна поведінка залежить від налаштування компілятора.

Обробка помилок виділення пам'яті – генерація виключення

```
try{
    int *ptr = new int[memsize];
}
catch(bad_alloc &e)
{
    cout << "Allocation error! Message: "
         << e.what() << endl;
}
```

Обробка помилок виділення пам'яті – повернення нульового вказівника

```
int *ptr = new(nothrow) int[memsize];
if (!ptr)
{
    cout << "Allocation error!" << endl;
}
```

new_handler

- std::new_handler – вказівник на функцію, що буде викликатися, якщо виділення пам'яті не вдалося.
- Може забезпечити наявність пам'яті, завершити програму std::terminate або кинути виключення std::bad_alloc чи його нащадка.
- Якщо встановити в nullptr, виконується поведінка за замовчуванням – кинути std::bad_alloc.
- Якщо функція завершилася – функція виділення пам'яті зробить ще одну спробу виділити пам'ять. Один із способів розірвати це коло – викликати std::set_new_handler(nullptr).
- Керування:

```
std :: new_handler set_new_handler( std::new_handler new_p ) noexcept;
std :: new_handler get_new_handler() noexcept;
```

Засоби C для маніпуляції динамічною пам'яттю

Прототипи функцій (файл <cstdlib>):

```
void* malloc (size_t size);
void free (void* ptr);
void* realloc (void* ptr, size_t size);
void* calloc (size_t num, size_t size);
```

Увага: вони нічого не знають про конструктори та деструктори C++.

malloc

Функції передається потрібна кількість **байт**, вона повертає вказівник на виділену область, або нуль — нульовий вказівник, якщо виділення неможливе.

```
void* malloc (size_t  size );
```

Виділення пам'яті:

```
double *d_ptr;
d_ptr = static_cast<double *> ( malloc( sizeof(double) * 15) );
if (d_ptr == NULL)
{
    cout << "Not enough memory!" << endl;
    exit (1);
}

  ..... Using memory ....

free (d_ptr);
```

Оскільки, зазвичай, пам'ять виділяємо під якісь конкретні змінні, то розмір в байтах обчислюється так: `sizeof(type) * N`, де type — тип змінних, N — їх необхідна кількість.

- Щодо розміру, N може дорівнювати нулю, але поведінка malloc при такому — на розсуд реалізації (implementation defined). Може повертати як нульовий, так і не нульовий вказівник, який, однак, заборонено розіменовувати.
- Більш деталізоване приведення вказівників:

```
void *v_ptr = malloc( sizeof(double) * 15);
double *d_ptr = (double *)v_ptr; // C—style cast
```

- `https://danluu.com/malloc-tutorial/` – простий приклад реалізації.

free

Для звільнення виділеної пам'яті використовується така функція:

```
void free (void* ptr);
```

Їй можна передавати вказівники, виділені malloc, calloc або realloc, чи нульовий вказівник (для якого вона просто нічого не робитиме). Будь які інші вказівники передавати заборонено — це призводить до undefined behaviour.

calloc

```
void* calloc (size_t num, size_t size );
```

Виділяє блок із num елементів, розміром size кожен, встановлює всі біти рівними 0. Поведінка за нульового розміру така ж, як і в malloc.

realloc

```
void* realloc (void* ptr, size_t size);
```

- Змінює розмір блоку, на який вказує **ptr**, повертає вказівник на новий блок розміру **size**.
- Якщо розмір блоку збільшується, значення, які вже були, зберігаються, решту виділеної області ніяк не ініціалізується[3].
- Якщо ptr нульовий, поводить себе як malloc — просто виділяє пам'ять.
- Якщо виділити новий блок не вдалося, повертає нульовий вказівник. При тому, блок, на який вказує ptr залишається незмінним.
- Якщо size=0, в C89, звільняє блок, повертає нульовий вказівник. На жаль, C99+ перестало давати такі гарантії. Як себе веде C++ – слід дивитися у відповідний стандарт...

[3] Тобто, там залишається сміття, звертання до якого – невизначена поведінка.

```
int x, sum=0;
size_t i, count = 0;
size_t block_size=5; /* initial block size */
int* data = static_cast<int*>(malloc( block_size * sizeof(int) ));

do {
    cout << "Enter non-zero number or 0 to exit: ";
    cin >> x;
    data[count]=x;
    ++count;

    if (count==block_size) {
        int* new_data = NULL;
        block_size *= 2;
        new_data = (int*) realloc (data, block_size * sizeof(int));

        if (new_data!=NULL) {
            data=new_data;
        } else {
            free (data);
            cout << "Error (re) allocating memory" << endl;
            return 1;
        }
    }

} while (x!=0);
```

Посилання

- Так може реальний malloc виглядати: `https://silent-tower.net/projects/visual-overview-malloc`. Єдине, sbrk зараз хіба в ембеддед трапляється, у інших випадках це за допомогою mmap роблять.

